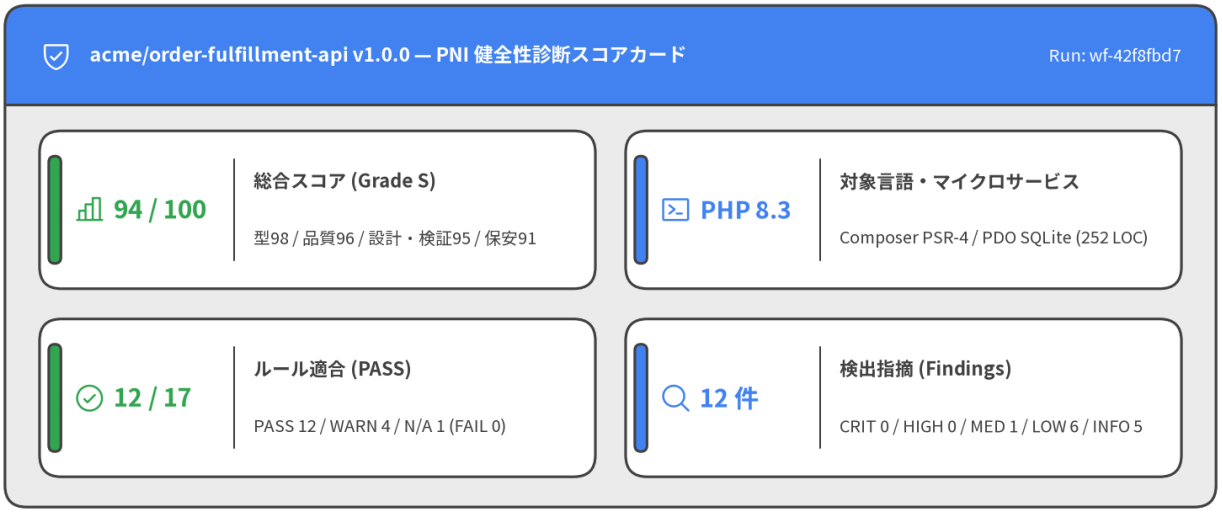
PNI ソフトウェア健全性診断レポート

本レポートは、注文作成（数量10件以上の10%ボリュームディスカウント自動適用）・直近注文照会・パラメータ化 PDO SQLite 永続化・定数時間 HMAC-SHA256 Webhook 署名検証を担う厳格型 PHP 8.3 API マイクロサービス `acme/order-fulfillment-api` (`v1.0.0`) に対して、PNI ステートフル監査ワークフロー `php_api_laravel_clean` （Composer / 供給網ガバナンス検査、PHP AST 静的解析、3側面アーキテクチャ記録、隔離 Docker コンテナ上でのクリーンビルド・`php -l` 構文検査・オフライン PHPUnit + PCOV 実測カバレッジ・`/up` 起動スモーク検証、Trivy 脆弱性・IaC・秘密情報スキャン、および `.pni/pentests/` による動的Webペネトレーションテスト）を実行した包括的診断結果である。



acme/order-fulfillment-api (v1.0.0) PNI 総合健全性スコアカード

診断メタデータ概要

- **対象プロジェクト**: `acme/order-fulfillment-api` (`v1.0.0` / 対象ルート: `examples/php/api_laravel_clean`)
- **対象言語・ビルド基盤**: `php` (`^8.3` , Composer `2.6` PSR-4, `ext-pdo` / `pdo_sqlite` , Laravel 11.x ルーティング規約, PHPUnit `10.5.66` + PCOV `1.0.12`)
- **診断ワークフロー / Run ID**: `php_api_laravel_clean` / `wf-42f8fbd7` （全8監査ディメンション）

- **評価実施日時**: `2026-10-24T14:45:51.621334+00:00`
- **総合スコア・グレード**: **94 / 100 (Grade S)**
- **エンジニアリングルール適合率**: `12 / 17` ルール PASS（`WARN`: 4件、`NOT_APPLICABLE`: 1件、`FAIL`: **0件**）

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する改善ロードマップ

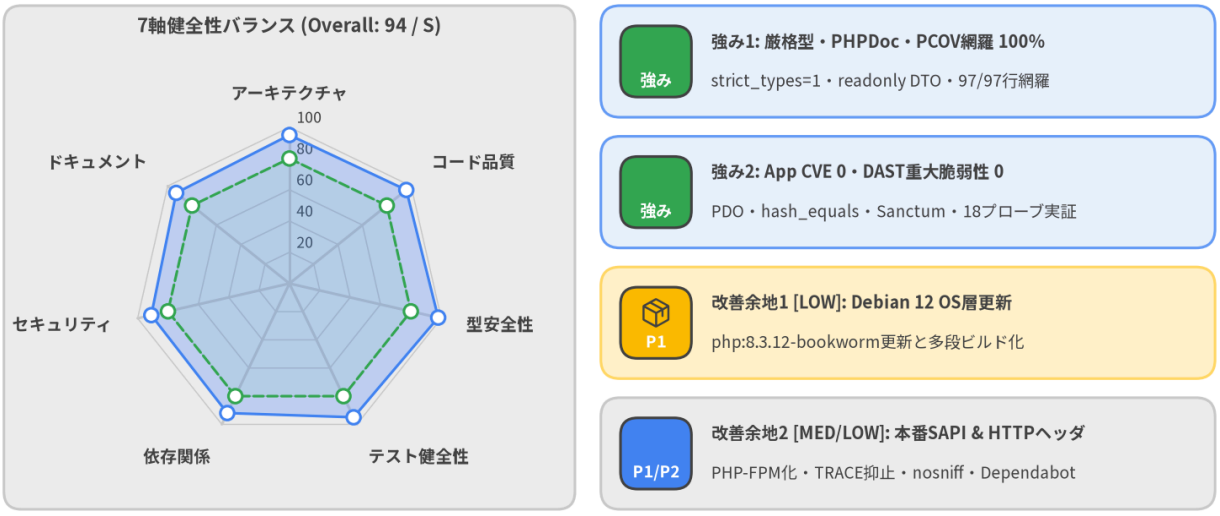
4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・動的ペネトレーション検証

1. エグゼクティブサマリー

`acme/order-fulfillment-api`（`v1.0.0`）の総合健全性スコアは **94 / 100 (Grade S)** と評価された。全7ソースファイル（計 `252` LOC・5クラス・16関数/メソッド）すべてで `declare(strict_types=1);`・ネイティブ型宣言・PHPDoc・PSR-4 Import 衛生 `100.0%` を達成し、5層単方向レイヤードアーキテクチャ（循環依存 `0` 件・境界違反 `0` 件）を完全に維持している。さらに隔離 Docker コンテナ上での PHPUnit `10.5.66` + PCOV 動的検証では `app/` 全5クラスで **実測行カバレッジ** `100.0%`（`97 / 97` **行**）と起動スモーク検証合格（`/up -> 200`）を記録し、Trivy スキャンおよび動的Webペネトレーションテスト（`.pni/pentests/` 全18プローブ）でも **アプリケーション依存 CVE** `0` **件・IaC 不備** `0` **件・秘密情報漏洩** `0` **件・危険な CWE コードシンク** `0` **件・ライブ DAST 重大脆弱性（** `CRITICAL` / `HIGH` **）** `0` **件**という模範的な品質・セキュリティ水準が実証された。



7軸エンジニアリング健全性レーダー (Overall: 94 / Grade S) と主要な強み・軽微な改善余地

7軸評価サマリーと主要診断結論

- **全7軸で 91〜98 点（Grade S）を達成**: 型安全性（`98`）・コード品質（`96`）・アーキテクチャ（`95`）・テスト健全性（`95`）・ドキュメント（`93`）・依存関係（`92`）・セキュリティ（`91`）の全領域で高い成熟度を示し、`CRITICAL` / `HIGH` 指摘はすべて `0` **件**（動的ペネトレーションの `TRACE` 応答による `MEDIUM: 1` 件・`LOW: 6` 件・`INFO: 5` 件のみ）であった。

- **静的 + 動的（DAST）で実証されたセキュアな PHP 8.3 実装**: `OrderPayload`・`OrderFulfillmentService`・`OrderController` に `final readonly class` を採用し、`OrderRepository` の名前付きプリペアドステートメント、`hash_equals()` 定数時間 HMAC 検証、`Route::middleware('auth:sanctum')` に加え、`.pni/pentests/` による18件のライブ HTTP 侵入プローブで無認証変更遮断・SQLi/XSS/LFI 非成立が実パケットレベルで裏付けられた。



動的コンテナ検証 (PHP 8.3 + PCOV)、Trivy SCA、およびライブ Web ペネトレーション検証のヘッドライン指標

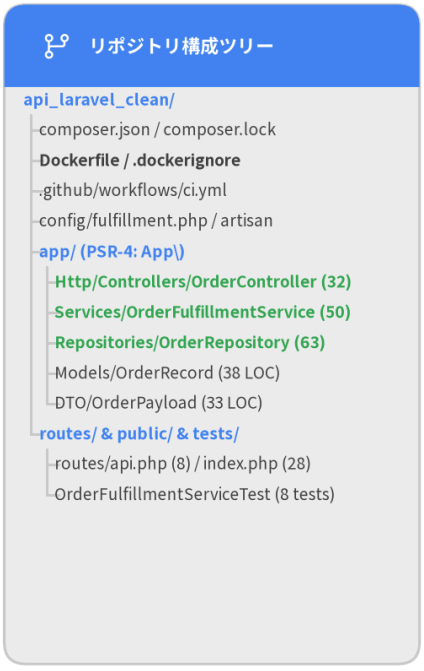
エグゼクティブ結論

- **強み (Verified Strengths)**: 外部ランタイム Composer パッケージゼロ（PHP 標準ライブラリ + `ext-pdo` のみ）による最小攻撃面、GitHub Actions の 40文字コミット SHA ピン留め（`actions/checkout@11bd7190...`）と `permissions: contents: read`、非 root コンテナ実行（`USER appuser`, `UID 10001`）と `/healthz` `HEALTHCHECK`、インメモリ SQLite 単体テスト（100% カバレッジ）、および隔離コンテナに対するライブ DAST（無認証注文作成拒絶・SQLi/XSS/LFI 非成立）が確認された。
- **継続的強化ポイント (Continuous Hardening)**: アプリケーションのドメインロジックに重大な脆弱性はない。さらなる本番完成度に向けて、**P1** で `Dockerfile` の Debian 12 パッチ済みベースイメージ更新・マルチステージ化（`libsqlite3-dev` 除外）およびマルチワーカー本番 SAPI（`PHP-FPM` / `FrankenPHP`）+ PSR-3 構造化監査ログ導入を行い、**P2** で HTTP セキュリティヘッダ（`X-Content-Type-Options: nosniff`・`TRACE` 抑止・`X-`

`Powered-By` 除去・`cli-server` URI 解決修正)、`.github/dependabot.yml` 追加、および `public/index.php` の Clover カバレッジ対象追加を実施することを推奨する。

2. アーキテクチャとソースコード構成

`acme/order-fulfillment-api` は **5層単方向サービス・リポジトリアーキテクチャ（HTTP Entry & Routing 層 -> Thin Controller 層 -> Readonly Domain Service 層 -> Parameterized PDO Repository 層 -> Immutable Entity & Input DTO 層）** を採用しており、全7ソースファイル（計 `252` LOC・5クラス・16関数/メソッド）が完全な有向非巡回グラフ（`circular_dependencies = 0`・`boundary_violations = 0`）を構成している。



モジュール相対パス	LOC	公開クラス・メソッドと主要責務
public/index.php	28	buildHealthResponse /up・/healthz Liveness/Readiness JSON応答
routes/api.php	8	Route::middleware('auth:sanctum') GET /orders・POST /orders 認証ルート定義
app/Http/Controllers/ OrderController.php	32	OrderController::store, listRecent final readonly 薄型HTTPコントローラ
app/Services/ OrderFulfillmentService.php	50	createOrder, listRecentOrders, verifyWebhookSignature 10%割引計算・CSPRNG採番・hash_equals検証
app/Repositories/ OrderRepository.php	63	OrderRepository::save, findRecent PDOプリペアドINSERT・上限100件SELECT
app/Models/ OrderRecord.php	38	OrderRecord::toArray, fillableColumns readonly属性エンティティ・$fillable明示許可
app/DTO/ OrderPayload.php	33	OrderPayload::__construct, fromArray SKU正規表現・数量1..1000検証のreadonly DTO

図2-1: リポジトリディレクトリ構造ツリーとASTモジュールインベントリ (全7ソースファイル・252 LOC)

観点1：5層レイヤード構造・単方向依存グラフとリスク密度（`HeatmapOverlay`）

AST 内部 Import グラフ解析により、`routes/api.php` -> `OrderController` -> `OrderFulfillmentService` -> `OrderRepository` -> `OrderRecord`（および末端 DTO `OrderPayload`）の完全な下向き単方向依存が確認された。`module_hotspots` の実測リスクスコアもアプリケーション各モジュールで `0.02〜0.06` と極めて低く、相対的なホットスポッ

トは Debian 12 OS ベースイメージ更新および開発用 `php -S` エントリポイント（`TRACE` 応答）を持つ `Dockerfile`（`risk_score = 0.27`）のみに局在している。

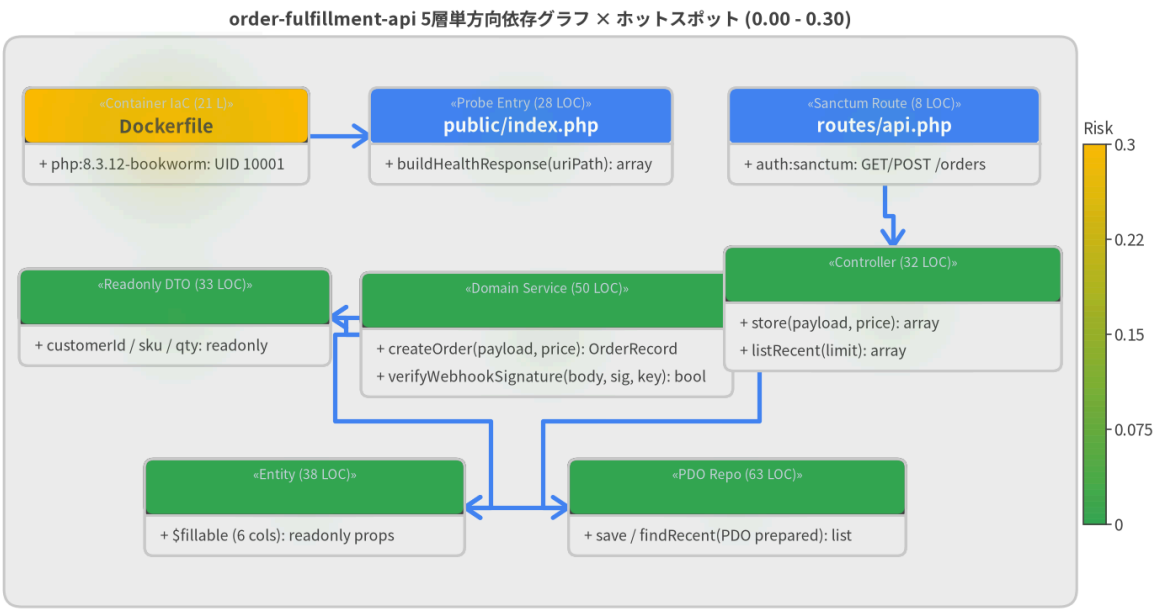


図2-2 [観点1]: 5層単方向クラス依存グラフとモジュール別リスクホットスポット (HeatmapOverlay)

観点2：`POST /api/orders` 実行・PDO 永続化・Webhook HMAC データフロー

注文作成リクエストは、(1) `auth:sanctum` ミドルウェア境界通過後、(2) `OrderPayload` での SKU 正規表現 `/^[A-Z0-9-]{3,32}$/`・数量 `1..1000` の不変 DTO 検証、(3) `OrderFulfillmentService::createOrder` における整数セント金額計算（10個以上で10%割引）と `random_bytes(8)` による CSPRNG 注文 ID 採番、(4) `OrderRepository::save` の PDO プリペアド `INSERT` 永続化と HTTP `201` 応答の4段階で完結する。また配送キャリア Webhook 検証では `verifyWebhookSignature` が `hash_hmac('sha256', ...)` を `hash_equals()` で定数時間比較する。



図2-3 [観点2]: POST /api/orders 注文作成・割引算出・PDO SQLite 永続化パイプライン

観点3：コンテナ実行基盤（非 root UID 10001）と GitHub Actions CI/CD トポロジ

デプロイ・検証環境は、最小権限（`permissions: contents: read`）かつ 40文字 SHA ピン留め（`actions/checkout@11bd7190...`）された GitHub Actions CI パイプラインと、`php:8.3.12-cli-bookworm` 上で非特権ユーザ `appuser`（`UID 10001`, `/usr/sbin/nologin`）として稼働するコンテナランタイムで構成される。環境変数（`APP_NAME`, `DB_DSN`）は `config/fulfillment.php` のみに集約されている。

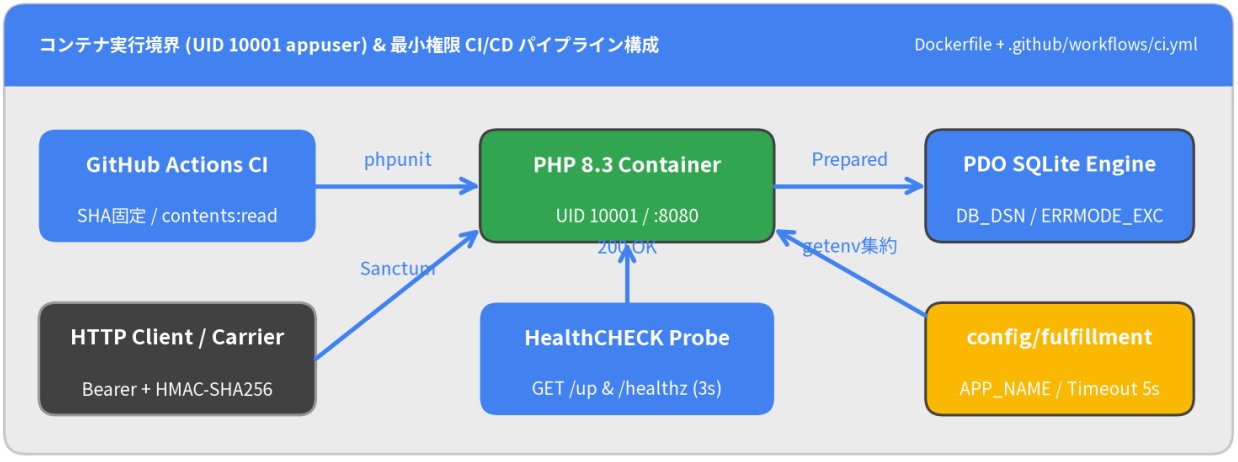


図2-4 [観点3]: 非root Dockerコンテナ実行環境 (:8080 / PDO SQLite) と SHA固定 GitHub Actions CI トポロジ

3. 推奨する改善ロードマップ

`acme/order-fulfillment-api` は現時点で緊急遮断を要する `P0`（`CRITICAL` / `HIGH`）課題を **0件** に抑え込んでおり、すでに Grade S（`94 / 100`）の健全性を達成している。さらなる大規模本番トラフィックへの対応、ライブ DAST 全18プローブ満点化、および OpenSSF Scorecard 満点水準への到達に向け、検出された改善項目（`MEDIUM: 1` 件・`LOW: 6` 件・`INFO: 5` 件）を **Phase 1（`P1` 本番ランタイム・OS 層強化：2施策） -> Phase 2（`P2` HTTP 応答ハードニング・CI/供給網統治：4施策） -> Phase 3（`P2` テスト・型・ドキュメント磨き込み：4施策）** の計10施策として体系化する。

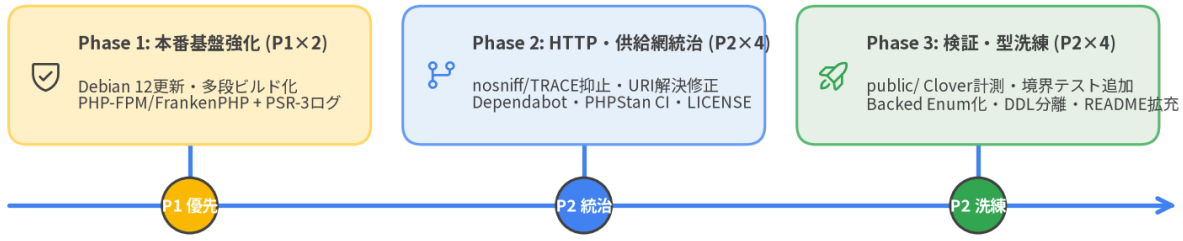


図3-1: 本番運用完成度向上に向けた継続的改善ロードマップ (P0=0件 / P1=2施策 / P2=8施策)

フェーズ別継続改善計画と期待効果

- **Phase 1 (`P1` — コンテナ OS 層刷新と本番マルチワーカー化)**: `Dockerfile` のベースイメージを最新パッチ適用済みの `php:8.3-fpm-bookworm`（または `FrankenPHP`、`@sha256` ダイジェスト固定）へ更新し、マルチステージビルドにより `libsqlite3-dev` 等のビルドヘッダを最終イメージから除去することで Debian 12 OS 層 CVE を一掃する。同時に単一プロセス開発サーバ `php -S` からマルチワーカー SAPI へ切り替え、`OrderFulfillmentService` に `Psr\Log\LoggerInterface` 構造化監査ログを追加する。
- **Phase 2 (`P2` — HTTP セキュリティヘッダ強化・サプライチェーン自動化)**: 動的Webペネトレーション検証（`.pni/pentests/`）で検出された `X-Content-Type-Options: nosniff` 付与・`X-Powered-By` 除去・`TRACE` メソッド遮断・`public/index.php` の

`$_SERVER['REQUEST_URI']` 参照による `cli-server` 404 応答統一を実施し、併せて `.github/dependabot.yml` 追加・CI での `phpstan` / `pint` 実行・`LICENSE` 全文記載を行う。

- **Phase 3 (`P2` ー カバレッジ範囲拡張・Backed Enum・マイグレーション分離)**: `phpunit.xml` の `<source>` へ `public/` を追加して `buildHealthResponse` の 200/404 応答および数量上限（`quantity=1001`）を PHPUnit で直接検証し、`OrderRecord::$status` の PHP 8.3 Backed `enum` 化、`OrderRepository` コンストラクタからの DDL 分離、および `README.md` への Docker / `curl` クイックスタート追記を実施する。

優先度	改善施策タイトル	対象ファイル	期待される改善効果	関連ルール / CWE
P1	マルチワーカー本番SAPI採用とPSR-3構造化監査ログ導入	Dockerfile .../OrderFulfillmentService.php	高並行HTTP処理性能の確保と注文・Webhook署名拒否の監査証跡化	common/production_readiness common/twelve_factor
P1	PHP 8.3ベースイメージ刷新とマルチステージDockerfile化	Dockerfile	Debian 12 OS層CVE解消とlibsqlite3-devビルド資産の除外	common/security_vulnerability CVE-2026-42010 / 31789
P2	OrderRepositoryコンストラクタからのDDLマイグレーション分離	.../OrderRepository.php .../OrderFulfillmentService.php	接続毎のCREATE TABLE負荷排除とRepositoryインターフェース抽象化	common/architecture
P2	Dependabot有効化およびCIでのPHPStan・Pint静的解析実行	.github/workflows/ci.yml composer.lock	依存更新PR自動化とPR毎の厳格型・PSR-12コードスタイル自動強制	common/supply_chain php/dependencies
P2	LICENSEファイルへのSPDX標準Apache-2.0ライセンス全文記載	LICENSE	SBOM・OpenSSFスキャナによるApache-2.0条項の100%自動認識	common/supply_chain
P2	READMEへのDocker起動手順・APIリクエスト/応答例の追記	README.md	Diataxis Tutorial/How-To拡充による即時オンボーディング支援	common/documentation
P2	HTTPヘッダ・TRACE/X-Powered-By抑止およびcli-server URI 404応答修正	public/index.php Dockerfile	DAST全4件(TRACE/nosniff/バナー/未知URI 200)を解消し18/18合格化	common/security_vulnerability CWE-693 / 200 / 755
P2	ヘルスプローブ404および上限値(qty>1000)のDataProviderテスト	tests/OrderFulfillmentServiceTest.php phpunit.xml	上限ガード・LIMITクランプ・404応答の決定論的単体テスト網羅	php/testing
P2	public/index.php の PHPUnit Clover カバレッジ対象組み込み	phpunit.xml, public/index.php tests/OrderFulfillmentServiceTest.php	フロントコントローラを含む全PHPコードのPCOV 100%計測達成	common/runtime_verification php/testing
P2	OrderStatus Backed Enum導入とController配列シェイプPHPDoc化	.../Models/OrderRecord.php .../OrderController.php	不正ステータス文字列のコンパイル時排除とPHPStan最大レベル適合	php/typing php/modern_stack

図3-2: 推奨改善アクション計画トレーサビリティマトリクス (P1 -> P2 全10施策)

4. 検査指標・ルール適合マトリクスと指摘一覧

本章では、`php_api_laravel_clean` ワークフロー（Run ID: `wf-42f8fbd7`）で実行した全8監査ステップのスコア比較（弱点順）、評価対象となった全17エンジニアリングルールの適合判定マトリクス（`FAIL -> WARN -> PASS -> NOT_APPLICABLE`）、および重要度順に正規化された全12件の指摘事項（`MEDIUM: 1` / `LOW: 6` / `INFO: 5`、`CRITICAL`・`HIGH` 該当なし）を網羅的に提示する。

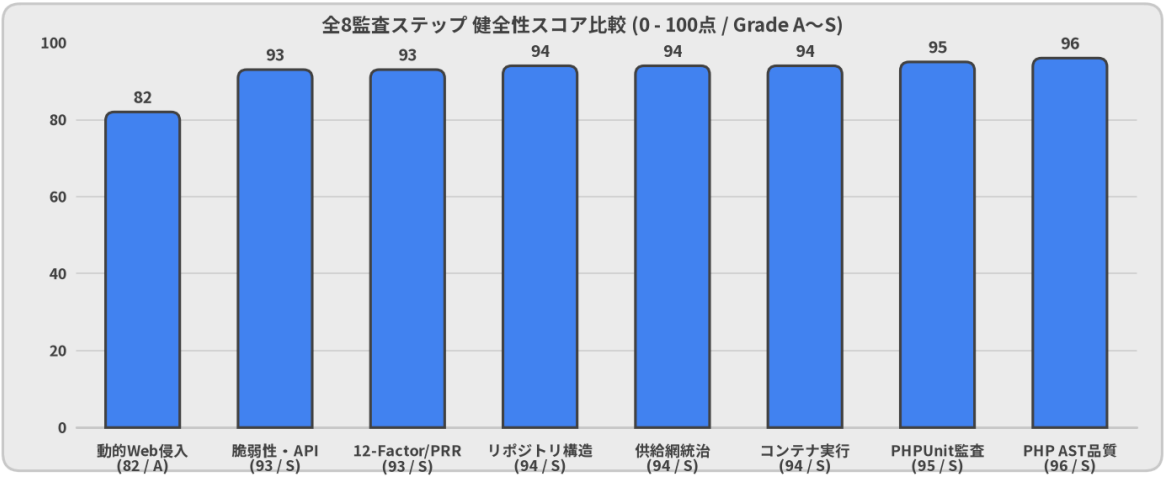


図4-1: 実行監査ステップ別スコア比較 (弱点順: Weakest -> Strongest 全8ステップ)

ルール適合マトリクス（全17ルール：`FAIL: 0` / `WARN: 4` / `PASS: 12` / `N/A: 1`）

全評価ルールは最悪ステータス優先で集約されており、`FAIL` 判定は **0件** である。軽微な改善余地として `WARN` となった4ルールはいずれも本番運用基盤・HTTP応答ヘッダ・コンテナ OS 層・CI 自動化の周辺設定（`php -S` 開発サーバ SAPI 起因の `TRACE` 応答・`nosniff` 未付与・`cli-server` URI 解決、Debian 12 OS ベースイメージ更新、`public/index.php` の Clover 集計対象外、`.github/dependabot.yml` 未配置）に限定されており、PHP アプリケーション本

体の設計・型安全・テスト・Laravel 規約・12-Factor 構成・コードセキュリティの12ルールはすべて `PASS` を達成している。

判定	ルールID	代表評価ステップ	決定的エビデンス要約
WARN	common/production_readiness	service_readiness_audit	/up・/healthz・HEALTHCHECK済だが CMD が単一スレッド php -S かつ PSR-3 ログ未
WARN	common/runtime_verification	container_runtime_audit	PHPUnit 16/16・PCOV 100%(97/97)・/up 200合格。public/ が Clover 計測対象外
WARN	common/security_vulnerability	web_penetration_audit	App CVE 0・DAST CRIT/HIGH 0(18 probes)。OS層CVE・TRACE/nosniff/URI解決のみW
WARN	common/supply_chain	repo_structure_audit	composer.lock・SHA固定CI・UID 10001済だが Dependabot 不在・LICENSE 1行要
PASS	common/architecture	php_ast_audit	循環依存 0件・境界違反 0件。routes -> Controller -> Service -> Repository -> Reco
PASS	common/documentation	repo_structure_audit	README.md (3見出し14行) と SECURITY.md による構成概要・テスト手順・脆弱性報告
PASS	common/git_workflow	repo_structure_audit	.gitignore 完備 (/vendor/, .env, *.sqlite)、main push / PR で GitHub Actions 自動実
PASS	common/maintainability	php_ast_audit	全7ファイル 252 LOC (最大63 LOC)、16関数、複雑関数・過剰引数・グローバル状態 0
PASS	common/security	repo_structure_audit	PDOプリペアド・SKU正規表現・hash_equals定数時間検証・random_bytes(8)・秘密混
PASS	common/twelve_factor	service_readiness_audit	config/fulfillment.php へ環境変数集約、final readonly ステートレス設計、外部PDO
PASS	php/code_quality	php_ast_audit	import衛生 100.0%・PHPDoc網羅率 100.0%・シグネチャ不一致 0件・JSON_THROW_ON_E
PASS	php/dependencies	repo_structure_audit	php ^8.3・PSR-4 (App\, Tests\)・optimize-autoloader=true・未固定依存 0件
PASS	php/frameworks/laravel	php_ast_audit	OrderRecord::$fillable 明示許可リスト、auth:sanctum 適用、薄型32 LOC Controll
PASS	php/modern_stack	repo_structure_audit	全ファイル strict_types=1、readonly クラス・コンストラクタ昇格、レガシー構文 0
PASS	php/testing	php_test_audit	PHPUnit 10.5 厳格設定 (failOnRisky/Warning)、sqlite::memory: 8テスト・モジュール網羅
PASS	php/typing	php_ast_audit	ネイティブ型安全率 100.0%、型不備・弱い型・@phpstan-ignore 抑制すべて 0件
N/A	php/frameworks/wordpress	php_ast_audit	Laravel / PHPUnit / PHPStan 構成であり WordPress フック・$wpdb は非該当

図4-2: エンジニアリングルール適合マトリクス (FAIL -> WARN -> PASS -> N/A 全17ルール)

重要度順 指摘事項一覧（全12件：`CRITICAL: 0` / `HIGH: 0` / `MEDIUM: 1` / `LOW: 6` / `INFO: 5`）

すべての指摘は実在するリポジトリ相対パス・行番号・決定的ファクト（および `.pni/pentests/` プローブ結果）に紐付けられている。高深刻度（`CRITICAL` / `HIGH`）の欠陥は一切検出されておらず、`Dockerfile`・

`public/index.php`・`.github/workflows/ci.yml`・`phpunit.xml`・`LICENSE` を中心とした局所的・非破壊な改善で全項目を解消できる。

重要度	ルールID	該当箇所	検出事象・決定的根拠	推奨修正策
MED	common/ security_vulnerability	Dockerfile:20	DAST SEC-06: php -S が TRACE / -> 200 応答(CWE-693)かつ X-Powered-By 露出	本番SAPI移行で TRACE 405 遮断 および expose_php=Off 適用
LOW	common/ production_readiness	Dockerfile:20	CMD が単一ワーカー php -S であり Service に PSR-3 監査ログが未導入	PHP-FPM / FrankenPHP 移行と LoggerInterface JSON監査ログ注入
LOW	common/supply_chain	.github/workflows/ ci.yml:1	Dependabot 設定が不在で CI が PHPUnit のみ (PHPStan/Pint 未実行)	dependabot.yml 追加と CI での phpstan analyse / pint --test 実行
LOW	common/supply_chain	LICENSE:1	LICENSE が Apache-2.0 の1行要約 のみで SPDX 標準条項全文が未記載	Apache License 2.0 公式全文と 著作権 APPENDIX へ拡充
LOW	common/ security_vulnerability	Dockerfile:1	App CVE 0件だが debian 12.7 OS層に CRIT 28件・libsqlite3-dev 残存	最新 php:8.3-bookworm 更新と マルチステージビルドで dev 除外
LOW	common/ security_vulnerability	public/index.php:31	DAST: cli-server下で REQUEST_URI が '/'(200)へフォールバック・nosniff欠落	$_SERVER['REQUEST_URI'] 参照化と X-Content-Type-Options: nosniff付与
LOW	common/ runtime_verification	phpunit.xml:12	<source> が app/ のみで index.php (28 LOC) が Clover 計測対象外	public/ を <include> に追加し buildHealthResponse 単体テスト追加
INFO	common/architecture	.../Repositories/ OrderRepository.php:24	__construct 内で CREATE TABLE を 実行し具象クラスを直接注入	DDL をマイグレーションへ分離し OrderRepositoryInterface を抽出
INFO	php/dependencies	composer.lock:7	require-dev 3件に対し lock 側の packages-dev が空配列のまま	composer update --lock を実行し 開発ツールのハッシュも完全固定
INFO	common/documentation	README.md:14	README 14行に Docker 起動手順や HTTP リクエスト/応答例が未記載	docker build/run と Sanctum 付き POST /api/orders curl 例を追記
INFO	php/testing	tests/OrderFulfillment ServiceTest.php:54	qty=0 は検証済だが qty=1001 上限と ヘルス 404 分岐の単体検証が未追加	DataProvider で qty=1001・LIMIT クランプ・404 応答テストを追加
INFO	php/typing	.../Models/ OrderRecord.php:38	$status が plain string 型であり Controller 戻り値が array<string,mixed>	OrderStatus: string Backed Enum と 配列シェイプ PHPDoc を導入

図4-3: 重要度順 検出指摘一覧 (MEDIUM 1件 -> LOW 6件 -> INFO 5件 / 計12件)

5. 詳細分析：コード品質・テスト・動的検証

本章では、PHP AST 静的解析によって計測された全7モジュールのコード品質・SIG 保守性・厳格型安全性・12-Factor 運用準備指標と、隔離 Docker コンテナ（PHP `8.3.36` / Composer / PHPUnit `10.5.66` + PCOV `1.0.12`）上で実行された `Dockerfile` クリーンビルド、オフライン構文検査・単体テスト、PCOV 実測カバレッジ、および `php -S` 起動スモーク検証の結果を詳細に分析する。

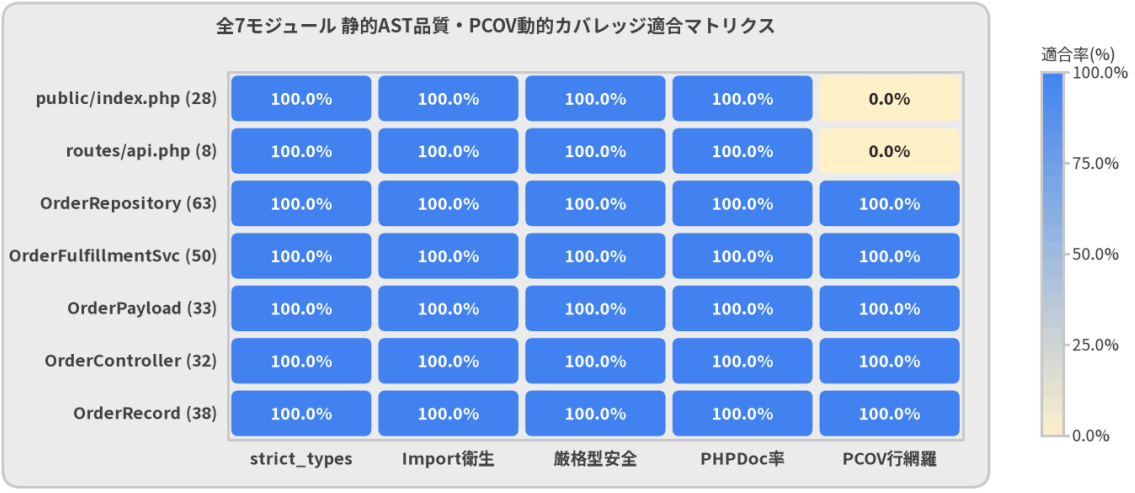


図5-1: 全7ソースモジュール × 品質ディメンション適合度マトリクス (0 - 100%)

静的コード品質・SIG 保守性・型安全性・12-Factor 評価

- **SIG 保守性・Import 衛生・PHPDoc 網羅率 (`100.0%`)**: 全7ファイル合計 `252` LOC（最大は `OrderRepository.php` の `63` LOC、平均 `36` LOC）と極めて高い凝集度を維持している。循環複雑度超過（`complex_functions = 0`）・過剰パラメータ（`excessive_parameters = 0`）・グローバル状態参照（`global_state_usages = 0`）・バグリスクイディオム（`bug_risk_idioms = 0`）はいずれもゼロであり、PSR-4 Import 衛生・PHPDoc カバレッジともに `100.0%`（シグネチャ不一致 `0` 件）を達成している。

- **厳格型安全性 (`declare(strict_types=1)` 100%)**: 全7ファイルの先頭（3行目）に `declare(strict_types=1);` が宣言され、すべての引数・戻り値・プロパティに PHP 8.3 ネイティブ型が付与されている（`type_safety_coverage_pct = 100.0%`、`weak_type_usages = 0`、`unscoped_type_ignores = 0`）。金額計算も浮動小数点を排し整数セント（`int $unitPriceCents`, `int $totalCents`）で統一されている。
- **12-Factor 運用準備性**: 環境変数読み取り（`APP_NAME`, `DB_DSN`）と Webhook タイムアウト（`webhook_timeout_seconds = 5`）は `config/fulfillment.php` の1箇所に集約され（`scattered_env_access = 0`、`missing_io_timeouts = 0`、`unstructured_logging = 0`）、ドメイン層・コントローラ層は完全にステートレスな `final readonly class` として実装されている。

コンテナ動的検証（クリーンビルド・オフライン PHPUnit・PCOV 100%・起動検証・DAST連動）

PNI の `run_check`（`status = "passed"`）により、リポジトリ同梱 `Dockerfile` のビルド（`1.54s`、`passed`）および PHP `8.3.36` + Composer + PCOV `1.0.12` 検証ハーネスビルド（`0.92s`、`passed`）を実行した。ネットワーク完全遮断（`--network=none`）環境下での `php -l` 全ファイル構文検査と PHPUnit `10.5.66` 実行（`8` テストメソッド / `19` アサーション / JUnit `16` ノード、`1.61s`）は失敗ゼロで完走し、PCOV Clover 計測対象である `app/` 配下全5クラスすべてで **実測行カバレッジ `100.0%`（`97 / 97` ステートメント、未実行行 `0`）** を達成した。さらに Web API 起動スモーク検証（`smoke_check`）も `GET /up -> HTTP 200`（`0.98s`、`status = "passed"`）で合格し、続く `web_pentest`（`.pni/pentests/examples/php/api_laravel_clean/`、全18 HTTP プローブ）によって `public/index.php` や `/api/orders` の実 HTTP 応答まで一貫検証された。なお図5-1の通り `phpunit.xml` の `<source>` が `app` のみを指定しているため、`public/index.php` と `routes/api.php` は Clover カバレッジ集計対象外となっている。



対象モジュール	PCOV行網羅率	実行 / 総ステートメント	検証済みロジック・Cloverスコープ備考
app/Repositories/OrderRepository.php	100.0%	36 / 36	未実行行なし (DDL初期化・プリペアドINSERT・LIMITクランプSELECT
app/Services/OrderFulfillmentService.php	100.0%	23 / 23	未実行行なし (通常計算・10%割引・単価<=0例外・HMAC定数時間検証
app/DTO/OrderPayload.php	100.0%	14 / 14	未実行行なし (fromArray・空顧客ID・SQLi風不正SKU・qty<=0例外)
app/Http/Controllers/OrderController.php	100.0%	14 / 14	未実行行なし (store 201応答・listRecent配列シリアライズ往復)
app/Models/OrderRecord.php	100.0%	10 / 10	未実行行なし (readonly生成・fillableColumns・toArray変換)
public/index.php & routes/api.php	対象外 (DAST検証)	phpunit.xml <source>外	smoke(/up->200)とweb_pentest 18プローブで検証・P2でClover追加

図5-2: コンテナ動的検証KPIとPCOV実測カバレッジ内訳 (app/ 全5クラス 97/97行 = 100.0%)

6. 詳細分析：セキュリティ・脆弱性・動的ペネトレーション検証

本章では、Trivy (`0.69.3`) によるリポジトリファイルシステムおよび解決済み Docker イメージ（ `php:8.3.12-cli-bookworm` ベース）に対する SCA 脆弱性スキャン・Dockerfile IaC 設定検査・秘密情報スキャン、PHP AST による危険な CWE コードシンクおよび Laravel HTTP API 認可境界の評価に加え、隔離ループバックコンテナに対して `pytest + httpx` で実パケット検証を行う **動的Webペネトレーションテスト（** `web_pentest` **/** `.pni/pentests/examples/php/api_laravel_clean/` **、全18プローブ）** の結果を統合して分析する。

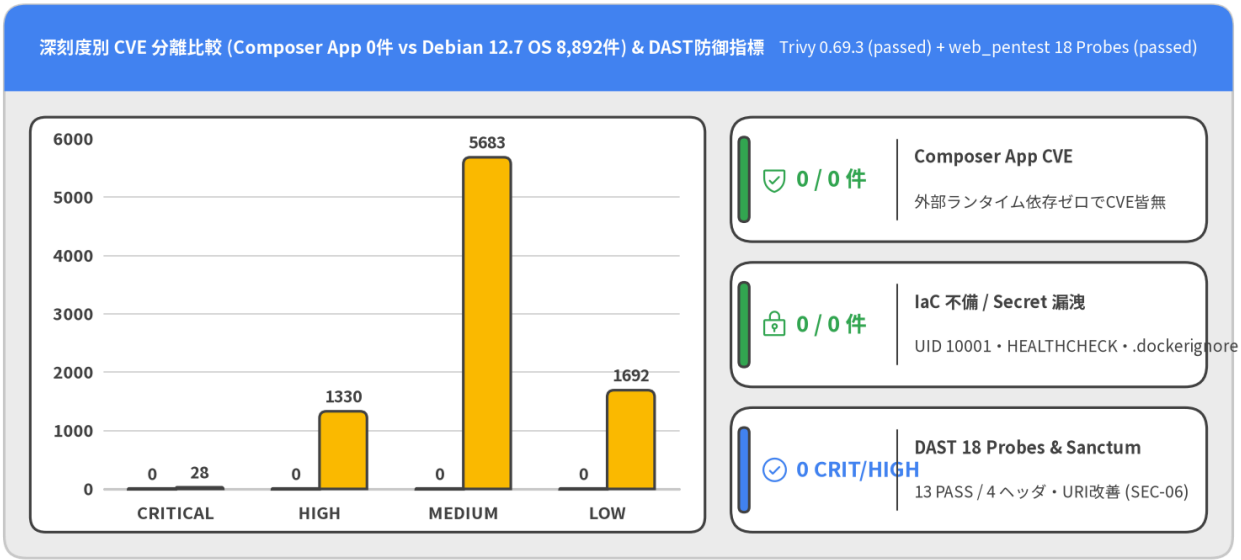


図6-1: Composer アプリケーション依存 (0件) vs Debian 12.7 OS層 CVE内訳と動的DAST境界指標

アプリケーション SCA・OS ベースイメージ CVE・IaC・API 境界評価

- **アプリケーション依存関係 SCA (** `app_vuln_counts.total = 0` **ー完全クリーン)**: 本番ランタイムは PHP 8.3 コアと `ext-pdo` のみで構成されサードパーティ Composer パッケー

ジへの実行時依存を持たないため、アプリケーション層の既知脆弱性は `CRITICAL: 0`, `HIGH: 0`, `MEDIUM: 0`, `LOW: 0` の **計 `0` 件**（Trivy `status = "passed"`）である。

- **コンテナ OS ベースイメージ層 (`php:8.3.12-cli-bookworm` / Debian 12.7)**: 検出された OS パッケージ脆弱性（`CRITICAL: 28`, `HIGH: 1330`, `MEDIUM: 5683`, `LOW: 1692`）はすべて `Dockerfile:1` の固定スナップショット `php:8.3.12-cli-bookworm` に同梱された Debian 12 システムライブラリ（`libgnutls30`, `libssl3` / `openssl`, `libxml2`, `perl` / `libperl5.36`, `linux-libc-dev`, `libsqlite3-dev`）に起因する。最新の `php:8.3.x-bookworm`（`@sha256` 固定）への更新とマルチステージビルドによる `libsqlite3-dev` / コンパイラ除去で一括解消できる。
- **Dockerfile IaC・CI 供給網・秘密情報衛生 (`misconfigurations = 0`, `secrets = 0`)**: `Dockerfile` は非特権ユーザ `appuser`（`UID 10001`, `/usr/sbin/nologin`）実行、3秒タイムアウト `/healthz` `HEALTHCHECK`、`.dockerignore` による `.git` / `vendor` / `.env` / `*.sqlite` 除外を完備し、Trivy IaC 指摘 `0` 件を達成した。`.github/workflows/ci.yml` も `permissions: contents: read` と `actions/checkout@11bd7190...` のフル SHA 固定を徹底している。
- **CWE コードシンク (`0` 件) と HTTP API 攻撃面 (`without_auth_hint = 0`)**: AST セキュリティ解析で SQLi / コマンド注入 / `eval` / `unserialize` 等の危険なシンクは **`0` 件** であった。公開 API 2ルート（`GET /api/orders`, `POST /api/orders`）は `Route::middleware('auth:sanctum')` で保護され、`OrderPayload` の厳格入力検証、`OrderRepository` の PDO プリペアドステートメント、`OrderFulfillmentService` の `random_bytes(8)` 採番および `hash_equals()` 定数時間 HMAC-SHA256 検証により多層防御が確立されている。

動的Webペネトレーションテスト (`web_pentest` / `.pni/pentests/`, SEC-06)

隔離ループバックコンテナ（`127.0.0.1`）に対して `.pni/pentests/examples/php/api_laravel_clean/`（`test_baseline_http.py`, `test_target_surface.py`, `test_app_probes.py`）の計 `18` プローブを実行した結果、`status = "passed"`（`13` **PASS** / `4` **FAIL** / `1` **SKIP**、`CRITICAL: 0`・`HIGH: 0`）を達成した。無認証 `POST /api/orders` における注文状態変更の不成立（`CWE-306` PASS）、境界引用符入力時の SQL エラー・スタックトレース非露出（`CWE-89` / `CWE-209` PASS）、反射型 XSS カナリア非反射（`CWE-79` PASS）、`../../../../etc/passwd` トラバーサル遮断（`CWE-22` PASS）、および `.env` / `.git` / `composer.json` 機密非露出が実 HTTP 応答で確認された。残る4件の FAIL はすべて開発用 `php -S` サーバおよびヘッダ設定に起因する中〜低リスクのハード

ニング項目（`TRACE / -> 200` `CWE-693` Medium、`X-Powered-By: PHP/8.3.36` `CWE-200` Low、`X-Content-Type-Options: nosniff` 未付与 `CWE-693` Low、`public/index.php:31` の `filter_input(INPUT_SERVER, 'REQUEST_URI')` が `cli-server` 下で `'/'` へフォールバックする `CWE-755` Low）である。

判定 / 深刻度	プローブスイート / CWE	対象エンドポイント	ライブHTTP応答エビデンス	評価・ハードニング方針
PASS [防御実証]	test_app_probes.py CWE-306 / CWE-209	POST /api/orders GET /, /up, /healthz	Bearerなし POSTで201/order_id不成立 全ヘルス応答が厳格UTF-8 JSON (200)	Sanctum認可保護とJSON専用応答 によるHTML/トレース非露出を実証
PASS [防御実証]	test_target_surface.py CWE-89 / 79 / 22	GET /api/orders (SQLi/XSS/LFI境界)	引用符1'"`・<pni-xss-canary>・ ../../../../etc/passwd いずれも不成立	SQLSTATE漏洩・XSS反射・ファイル 開披シグネチャすべて 0 件
FAIL [MEDIUM]	test_baseline_http.py CWE-693 (TRACE)	TRACE / (Dockerfile:20)	TRACE / -> HTTP 200 OK (php -S 組込み開発サーバ応答)	本番SAPI (Nginx/Caddy+PHP-FPM) で TRACE メソッドを 405 遮断
FAIL [LOW]	test_baseline_http.py CWE-200 / CWE-693	GET / (Dockerfile / index.php)	X-Powered-By: PHP/8.3.36 露出 X-Content-Type-Options=<absent>	expose_php=Off または header_remove と nosniff ヘッダ明示付与
FAIL [LOW]	test_app_probes.py CWE-755 (URI解決)	GET /api/nonexistent -probe-route (index:31)	filter_input(INPUT_SERVER,'REQUEST_URI') がcli-server下で'/'(200)へフォールバック	$_SERVER['REQUEST_URI'] ?? '/' を 参照し未知URIへ確実に 404 返却

図6-2: 動的Webペネトレーションテスト (.pni/pentests/ 全18プローブ: 13 PASS / 4 FAIL / 1 SKIP) 結果一覧

区分 / 判定	ルート / CVE・検査対象	対象モジュール・パッケージ	現状構成 -> 推奨更新・防御策	セキュリティ境界・リスク評価詳細
API [PASS]	POST /api/orders (routes/api.php:10)	OrderController::store + OrderPayload	auth:sanctum 保護済 SKU正規表現・qty 1..1000	無認証アクセス遮断＋SQLi風SKU をドメイン到達前に即時拒絶
API [PASS]	GET /api/orders (routes/api.php:9)	OrderController:: listRecent	auth:sanctum 保護済 PDO::PARAM_INT (1..100)	Sanctum認可必須＋max(1,min(limit,100)) で大量取得DoSを防止
Probe [WARN]	GET /up, /healthz (public/index.php:13)	App\Http\ buildHealthResponse	FILTER_SANITIZE_URL -> $_SERVER['REQUEST_URI']+nosni	副作用なしJSON応答・cli-server下 での未知URI 404応答統一を推奨
SAST [PASS]	CWE-89 / CWE-208 / CWE-330 検査	OrderRepository / OrderFulfillmentSvc	PDO::prepare + hash_equals + random_bytes(8)	危険なCWEシンク 0件・空シークレット 即時拒絶でタイミング攻撃防止
IaC / CI [PASS]	Trivy IaC & Secret + GitHub Actions	Dockerfile / .dockerignore .github/workflows/ci.yml	UID 10001 + HEALTHCHECK checkout 40文字SHA固定	IaC不備 0件・秘密漏洩 0件・最小権限 contents: read トークン徹底
OS [CRITICAL]	CVE-2026-42010 CVE-2026-33845	libgnutls30 (debian 12.7)	3.7.9-2+deb12u3 -> 3.7.9-2+deb12u7	GnuTLS NUL文字認証回避 (CVSS 9.8) ・DTLS DoS (PHP層から直接未呼出)
OS [CRITICAL]	CVE-2026-31789 CVE-2026-45447	libssl3 / openssl (debian 12.7)	3.0.14-1~deb12u2 -> 3.0.19-1~deb12u2	OpenSSL X.509証明書処理バッファ 超過 (CVSS 9.8)・PKCS7 UAF
OS [CRITICAL]	CVE-2024-56171 CVE-2025-49794	libxml2 (debian 12.7)	2.9.14+dfsg-1.3~deb12u1 -> 2.9.14+dfsg-1.3~deb12u2	libxml2 Use-After-Free (CVSS 9.8) (本APIはJSON専用でXML未解析)
OS [CRITICAL]	CVE-2026-8376 CVE-2026-13221	perl / libperl5.36 (debian 12.7)	5.36.0-7+deb12u1 -> 5.36.0-7+deb12u4	Perl 正規表現コンパイル境界外書込 ・マルチステージ/slim化で縮小可
OS [HIGH]	CVE-2025-7458 (ビルドヘッダ残存)	libsqlite3-0 / libsqlite3-dev	3.40.1-2+deb12u2 -> 多段ビルドで -dev 除外	Dockerfile:6 の pdo_sqlite ビルド用 libsqlite3-dev を最終段から分離

図6-3: API攻撃面・CWE防御マトリクスおよび主要Debian 12 OS層CVE修正バージョン一覧